Module 6 · SW Design

Secure Coding

Most breaches start in ordinary code. Secure coding is defensive design.

Akın Kaldıroğlu · Lead Trainer

Why it matters

Security Is Everyone's Job

Security isn't a team down the hall — it's a property of the code you write today. The cheapest bug to fix is the one you never write.

Never trust input

All external data is guilty until validated.

Fail securely

Errors must not leak secrets or open doors.

Least privilege

Give every part only the access it needs.

This is defensive programming with an adversary: the same barricade — but now the bad input is deliberate, and someone is probing for the gap you left.

What you'll leave with

Learning Objectives

- Tell validation, sanitization and encoding apart — and apply each in its place.
- Stop injection with parameterized queries and output encoding.
- Handle errors without leaking information — fail securely.
- Manage resources and secrets safely; apply defense in depth.

First, define it

What Is Security?

Security is not a feature you bolt on — it is a set of properties your system either keeps or loses. Four of them, remembered as CIA-T:

Confidentiality Only those who should see it, do.

Broken by: leaks, eavesdropping, over-broad access, verbose errors.

Integrity Data and code change only in intended ways.

Broken by: tampering, injection, unauthorized writes, silent corruption.

Availability The system is there when it is needed.

Broken by: overload, resource exhaustion, ransomware, a bad deploy.

Traceability You can tell who did what, and when.

Broken by: missing audit trails — nobody can prove, or disprove, anything.

Every vulnerability is one of these four failing. Security work is keeping them true even while someone attacks them.

Access control is the main lever, not the whole answer: *authentication and authorization enforce these properties — but so do encryption, data minimization and quiet error messages.*

Naming the threats

STRIDE — How You Lose Them

If CIA-T is what you want to keep, STRIDE is the checklist of ways to lose it — six threat categories, one per property (Microsoft, 1999):

S	**Spoofing**	— pretending to be someone or something else	**breaks Authentication**
T	**Tampering**	— altering data or code, in transit or at rest	**breaks Integrity**
R	**Repudiation**	— denying an action, with no way to prove otherwise	**breaks Traceability**
I	**Information disclosure**	— exposing data to the wrong eyes	**breaks Confidentiality**
D	**Denial of service**	— exhausting a resource until nobody can use it	**breaks Availability**
E	**Elevation of privilege**	— gaining rights you were never granted	**breaks Authorization**

Walk each part of your design and ask the six questions. Whatever you find becomes a requirement, a control, or a test.

Spoofing and elevation break the two properties that guard the other four: authentication (who you are) and authorization (what you may do).

The two guards

Authentication vs Authorization

The two words behind STRIDE's S and E — asked in this order, and routinely confused:

Authentication (authN)

Who are you?

Password, passkey, second factor, client certificate, mTLS between services, or a token carrying an identity already verified.

Fails as spoofing — someone acts as an identity that is not theirs.

Authorization (authZ)

What may you do?

Roles (RBAC), attributes (ABAC), access lists, OAuth scopes, policy engines, row-level security in the database.

Fails as elevation of privilege — a real identity exceeding its rights.

Asked	once, at the start of a session	on every single request
Gets wrong	someone else's identity	your identity, exceeding its rights

Most breaches live in that second row: apps authenticate carefully at login, then forget to authorize each action — change the id in the URL and you read someone else's order. Note HTTP 401 means unauthenticated; 403 is the authorization failure.

Section 01

Validate All Input

Every input is an attack until proven otherwise.

Input

Trust Nothing From Outside

Requests, files, headers, environment, other services — all of it is untrusted. Validate at the boundary, and prefer allow-lists.

Weak

- Block a few 'bad' characters (deny-list)
- Validate only on the client
- Assume the format is correct

Strong

- Allow only known-good input (allow-list)
- Validate on the server, always
- Check type, length, range, format

Input

Validate, Sanitize, Encode

Three different jobs, routinely confused. Each asks its own question, and each belongs in its own place.

Validate — is this acceptable?

A yes/no decision; the value is never changed. Invalid input is rejected. e.g. quantity must be an integer 1–100 → reject 0, –5, “ten”.

Sanitize — make it canonical or safe

The value is rewritten: trim spaces, normalize Unicode, lower-case an email domain, strip tags from a comment. Prefer rejecting over stripping — cleaning is guesswork.

Encode / escape — safe for where it is going

Applied at the destination, not the source: HTML-escape before rendering, parameterize before SQL. The same value is safe in one sink, dangerous in another.

Validation rejects · sanitization rewrites · encoding depends on the destination.

Order matters: *canonicalize first, then validate — otherwise %2e%2e%2f slips past a check for ../*

Input

Where Does Validation Live?

‘Validate at the boundary’ is only half the answer. The boundary checks the shape; the domain decides what is meaningful.

Edge — controllers, DTOs, parsers

Syntactic: is it well-formed? Type, length, format, encoding. Reject early, reject cheaply — a bad request never reaches the model.

Domain — entities & value objects

Semantic: is it meaningful? The invariants that must hold for the concept to exist at all. Enforced in the constructor, so an invalid object is unbuildable.

Everywhere else — the smell

The same if-checks repeated in service after service, because the value is still a String or a double. Miss one caller and the hole is open.

Edge validates the shape; the domain owns the meaning — then the rest of the code can trust its types.

📖 **Secure by Design — Deogun, Johnsson & Sawano** — *domain primitives: security as a design property, not bolted-on checks*

Input

Two Questions, Two Homes

Same input, two different questions — and they cannot be answered in the same place:

Input	Syntactic — at the edge	Semantic — in the domain
Email	format, ≤ 254 chars	not already registered; domain allowed
IBAN	34 chars, checksum valid	account exists; belongs to this customer
Quantity “12”	integer, 1–100	within stock and the credit limit
Currency “USD”	3 uppercase letters	ISO 4217; supported by this merchant
Date “2026-07-30”	valid ISO-8601 date	in the future; a working day

A parser or a framework annotation can answer the left column. Only the domain can answer the right — which is why those checks belong in entities and value objects.

Input

A Value Object Validates Itself

Money is not a number and a string — it is one concept with rules. Put the rules where the concept lives:

Java *the constructor is the only door in — no invalid Money can be built*

```
public final class Money {
  private final BigDecimal amount;
  private final Currency currency;

  public static Money of(String amount, String code) {
    Currency c = Currency.getInstance(code);   // ISO 4217 allow-list, not a regex
    BigDecimal v = new BigDecimal(amount);     // never double — rounding is a bug
    if (v.signum() < 0) throw new InvalidMoney("must not be negative");
    if (v.scale() > c.getDefaultFractionDigits())
      throw new InvalidMoney("too many decimals for " + code);
    return new Money(v, c);                    // valid, or it does not exist
  }
}
```

Currency validates against ISO 4217; amount validates against that currency's own scale.

Input

Invariants Travel With the Type

primitives — check everywhere

```
void transfer(String from, String to,
              double amount, String ccy) {
  if (amount <= 0)      throw ...;
  if (ccy.length() != 3) throw ...;
  // …and again in every other method
  //    that touches money
}
```

value object — check once

```
void transfer(Account from, Account to,
              Money amount) {
  from.withdraw(amount);   // nothing to
  to.deposit(amount);      // validate here
}

Money add(Money other) {        // invariant
  if (!currency.equals(other.currency))
    throw new CurrencyMismatch();  // preserved
```

Validation becomes a type, not a habit. Every method that takes Money is safe by signature — and operations keep the invariant: you cannot add euros to dollars by accident.

Input

▶ runnable `dev.kaldiroglu.bootcamp.secure.sqlinjection`

Never Build Queries by Concatenation

String-building puts the user in control of your SQL. Parameterize — always:

SQL injection

```
// Java — user controls the query!
"SELECT * FROM u WHERE name='"
   + name + "'";
// name = ' OR '1'='1  → dumps table
```

Parameterized

```
// Java — driver treats name as data
var ps = con.prepareStatement(
  "SELECT * FROM u WHERE name = ?");
ps.setString(1, name);
```

Same rule in every language: prepared statements / parameters (C# SqlParameter, Python DB-API params). Never concatenate.

Input

Sanitize Output, Too

Validation guards what comes in; encoding guards what goes out. Data safe in a database can still be dangerous when rendered.

- Encode for the destination: HTML, URL, SQL, shell — each differs.
- Cross-site scripting (XSS): unencoded user text becomes executable HTML.
- Use framework escaping; don't hand-roll it.

Section 02

Fail Securely

How your code fails is part of its attack surface.

▶ runnable dev.kaldiroglu.bootcamp.secure.errorleak

Don't Leak in Error Messages

Leaks internals

```
catch (SQLException e) {
  return e.getMessage()
    + stackTrace(e);   // to the user!
  // reveals schema, paths, versions
}
```

Safe

```
catch (SQLException e) {
  log.error("query failed", e); // server
  return "Something went wrong"; // user
  // detail stays internal
}
```

Log the detail where only you can see it; show the user a generic, honest message.

Errors

Exceptions, Not Silent Failure

Dangerous

- Empty catch blocks that swallow errors
- Returning null / -1 to signal failure
- Continuing in an invalid state

Safe

- Throw meaningful, specific exceptions
- Fail fast — stop before damage spreads
- Leave the system in a valid state

Section 03

Resources & Secrets

Clean up what you open; never hard-code what must stay secret.

Resources

Release What You Acquire

Leaked connections, files and memory cause outages — and outages are security incidents. Let the language guarantee cleanup.

- Use try-with-resources (Java), using (C#), with (Python) — deterministic cleanup.
- Bound everything: buffer sizes, upload limits, request timeouts.
- In manual-memory languages: validate lengths; avoid overflows and use-after-free.

Resources

▶ runnable dev.kaldiroglu.bootcamp.secure.secrets

Keep Secrets Out of Code

Never

```
String key =
  "sk_live_9f8a...";  // in source!
// leaks to git, logs, screenshots
```

Instead

```
String key =
  System.getenv("API_KEY");
// config / secret manager / vault
```

Secrets belong in environment config or a secret manager — never in the repository, logs, or error messages.

The mindset

Develop a Hacker Mind

A developer asks “does it work?” An attacker asks “what else does it do?” Secure code comes from holding both questions at once — you cannot defend a system you have never tried to break.

You think in use cases

- The user types a quantity
- The form validates it
- The order is placed

An attacker thinks in abuse cases

- I send quantity = –1
- I skip the form and POST directly
- I place 10,000 orders a second

Every assumption you never wrote down is a door you did not know you left open.

The mindset

How to Think Like an Attacker

It is a habit, not a talent — and it is built by asking the same questions every time you write a feature.

Ask while you code

- Who says this value is what it claims?
- What if I change the id to someone else's?
- What if I skip a step — or replay one?
- What if it arrives 10,000 times a second?
- What do my error messages give away?

Build the habit

- Write one abuse case per user story
- Run STRIDE over the design (Shostack)
- Read the OWASP Top 10 as a checklist
- Practice on Juice Shop, WebGoat, CTFs
- Review a PR wearing the attacker hat

OWASP — Open Worldwide Application Security Project: *a non-profit publishing vendor-neutral guidance. Its Top 10 ranks the most critical web application risks; its Cheat Sheet Series is the practical how-to.*
Attack only what you own or are invited to test — *the skill is legitimate; the target must be too.*

The mindset

Defense in Depth

No single control is enough. Layer them, so one failure doesn't become a breach.

Validate

Reject bad input at every boundary.

Least privilege

Minimal rights for each user, service, token.

Assume breach

Log, monitor, and limit the blast radius.

The AI angle

AI Can Introduce Vulnerabilities

Generated code often takes the insecure shortcut — string-built queries, swallowed errors, hard-coded keys. You are the security reviewer.

- Explicitly ask for parameterized queries and validated input.
- Review AI diffs for injection, secret leaks, and unsafe error handling.
- Never paste real secrets or production data into a prompt.

Keep in your kit

More Techniques · Crypto & Access

Beyond this session — the practices that catch what validation and error handling cannot:

Modern password hashing — *Argon2id first; bcrypt, scrypt or high-iteration PBKDF2 acceptable. Never MD5 or raw SHA-256. Salt every password.*

Don’t roll your own crypto — *Vetted libraries, authenticated encryption (AES-GCM), SecureRandom — never Random() — and constant-time comparison.*

Obscurity is not a control — *Hidden endpoints and renamed parameters are speed bumps. Assume the attacker reads your source (Kerckhoffs).*

Authorize every request, server-side — *Hiding a button is not access control. Broken access control is number one on the OWASP Top 10.*

Least privilege — *Minimal database grants, scoped tokens, and a separate account per service.*

Session & cookie hygiene — *HttpOnly, Secure, SameSite; rotate the session id on login; expire it sensibly.*

Keep in your kit

More Techniques · Boundaries & Operations

Most breaches today arrive through a dependency or a default nobody changed:

- **Fail closed, secure by default** — *Deny unless allowed. The safe path must be the default path, not the careful one.*
- **Limits everywhere** — *Max body size, pagination caps, timeouts, rate limits — against brute force, DoS and ReDoS.*
- **Minimize the attack surface** — *Turn off unused endpoints, features and debug modes before production.*
- **Your dependencies are your code** — *Audit them, lock versions, patch promptly, prefer fewer. Most CVEs arrive this way.*
- **Automate the checks** — *SAST, secret scanning and dependency alerts in CI — guardrails beat vigilance.*
- **Log security events, never secrets** — *No passwords, tokens or PII in logs. Do log auth failures and privilege changes.*
- **Encrypt in transit and at rest** — *TLS everywhere — and think about what is sitting in the database.*

CVE — Common Vulnerabilities and Exposures: *the public catalogue of flaws — CVE-2021-44228 is Log4Shell; scanners check yours.*

Run it yourself

The Code Behind This Session

Every example in this session compiles and runs — the same lessons in three languages, with the smell and the fix sitting side by side:

Java

Maven · JUnit 5 · 5 tests here

`secure/sqlinjection/`
VulnerableUserDao vs parameterized query

`secure/errorleak/`
an error that says nothing useful to an attacker

`secure/secrets/`
secrets from configuration, never source

C# / .NET

xUnit · 4 tests here

`Secure.cs`
injection · error leak · secrets

`SecureTests.cs`
the injection actually succeeds in the smell

Python

pytest · 4 tests here

`secure.py`
injection · error leak · secrets

`tests/test_secure.py`
concatenation is exploitable

`github.com/kaldiroglu/Agentic-Software-Design-and-Architecture-Bootcamp-JAVA · -DOTNET · -PYTHON`

The vulnerable version really is vulnerable. The test feeds it ' OR '1'='1 and watches the table come back — a demonstration, not a warning.

Clone one, run the suite, then break something and watch which test goes red. ***Across all chapters the suites are 70 · 63 · 65 tests.***

Remember these

Key Takeaways

- ✓ Never trust input — validate at the boundary with allow-lists.
- ✓ Parameterize queries; encode output. No string-built SQL or HTML.
- ✓ Fail securely: log detail internally, show users a generic message.
- ✓ Release resources deterministically; keep secrets out of code.
- ✓ Defense in depth — and treat AI-generated code as unreviewed.

Go deeper

Further Reading

The books behind this session. If you read only one, read the first.

📖 **Secure by Design** Deogun, Johnsson & Sawano · Manning, 2019

Security as a design property — value objects and invariants, not bolted-on checks.

📖 **Threat Modeling: Designing for Security** Adam Shostack · Wiley, 2014

How to find what can go wrong, systematically, before writing the code.

📖 **Writing Secure Code, 2nd ed.** Howard & LeBlanc · Microsoft Press, 2003

Still the clearest treatment of input validation, least privilege and secret handling.

📖 **The Web Application Hacker's Handbook, 2nd ed.** Stuttard & Pinto · Wiley, 2011

The attacker's view — read it to understand why the defenses are shaped as they are.

OWASP Top 10 & Cheat Sheet Series — the working reference → `owasp.org`

Next session

What's Next

Module 7

Object Stereotypes

with Akın Kaldıroğlu

Objects come in kinds. Three vocabularies for saying which kind you are looking at — and why the answer changes how you test it.

Further practice

Guard the Boundaries

Trace where untrusted input enters the ACME Orders system, and make it validate and fail safely.

Refactoring course · Defensive Programming

Repo `github.com/kaldiroglu/refactoring-course-student-materials`

IN THIS SESSION, LOOK AT

- `Slides-PDF/Ch08-DefensiveOffensive.pdf`
- `.../web/OrderController.java  (input boundary)`
- `.../util/Util.java  (unchecked helpers)`

Questions?

Assume nothing. Validate everything.

Akın Kaldıroğlu · Secure Coding